



PRACTICAL LOGBOOK

BIT3208: Advanced Web Design and Development

ShopMart — PHP/MySQL E-Commerce System (Weeks 1–5)

Student Name	Stephen Otiende
Admission Number	BBIT/2018/36596
Course / Class	Bachelor of Business Information Technology
Unit Code	BIT3208
Unit Name	Advanced Web Design and Development
Lecturer Name	Mr. Michael Nyoro
Semester / Year	Semester 2 / 2025–2026
GitHub / Portfolio	https://github.com/Otyzo/Advanced-web-design-Tasks

Contents

Week 1: Local Environment Setup	3
Title	3
What Was Built.....	3
Required Evidence	3
Student Reflection (Maximum 100 Words)	5
Tools Used.....	5

Week 2: Wireframes and GUI Design	6
Title	6
What Was Built.....	6
Required Evidence	6
Student Reflection (Maximum 100 Words)	9
Tools Used.....	9
Week 3: JavaScript and PHP Basics	10
Title	10
What Was Built.....	10
Required Evidence	10
Student Reflection (Maximum 100 Words)	13
Tools Used.....	14
Week 4: User Authentication.....	15
Title	15
What Was Built.....	15
Required Evidence	15
Student Reflection (Maximum 100 Words)	16
Tools Used.....	17
Week 5: Database Components.....	18
Title	18
What Was Built.....	18
Required Evidence	18
Student Reflection (Maximum 100 Words)	19
Tools Used.....	20

Week 1: Local Environment Setup

Installation and Testing of Local Development Environment

Title

Installation and Testing of Local Development Environment

What Was Built

The local development environment for ShopMart was installed and verified using XAMPP, which provides Apache, MySQL, and PHP. Three small PHP scripts were created to confirm each layer of the stack: a Hello World page confirming Apache executes PHP correctly, a database connection script confirming PHP can reach MySQL via mysqli, and a JSON test endpoint used in Postman to inspect HTTP requests and responses directly.

Required Evidence

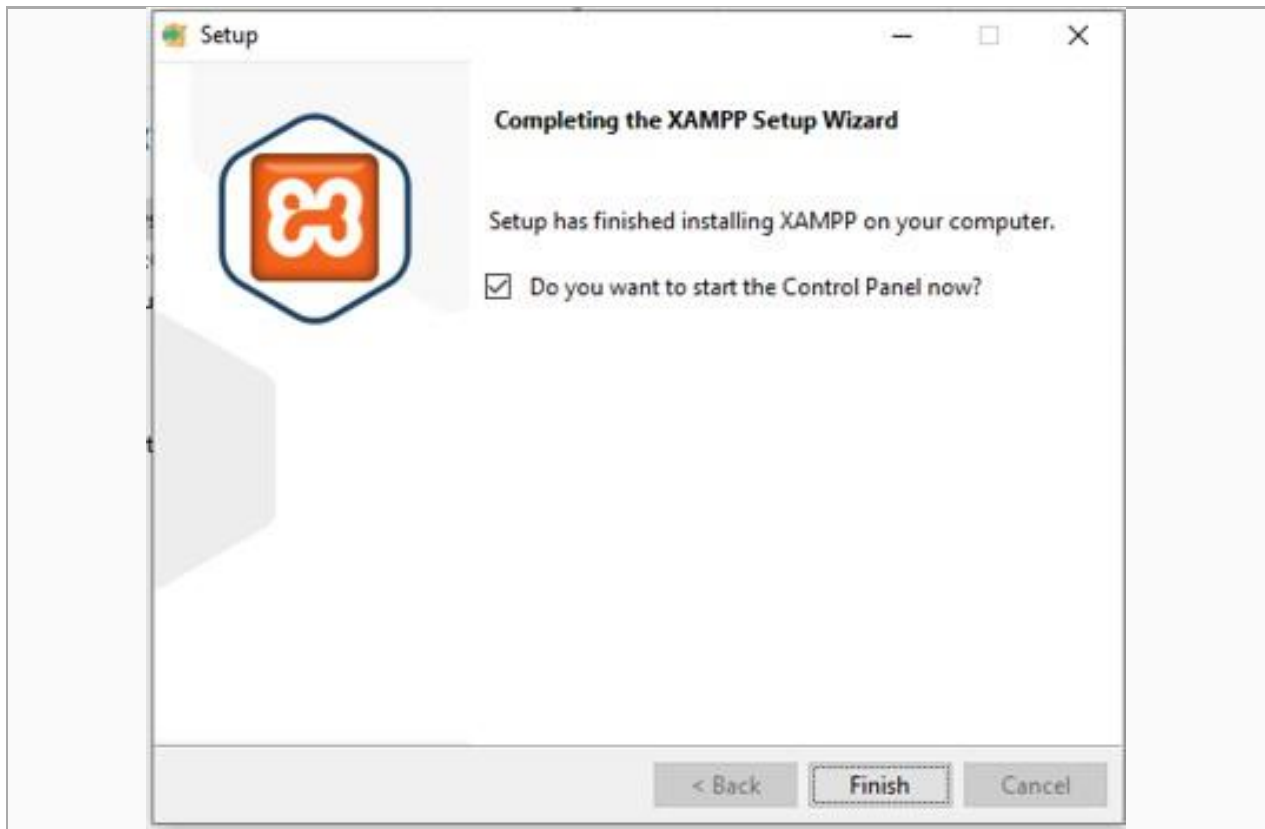


Fig 1: XAMPP Installation Complete — The XAMPP setup wizard confirming successful installation on Windows, with the option to launch the Control Panel.

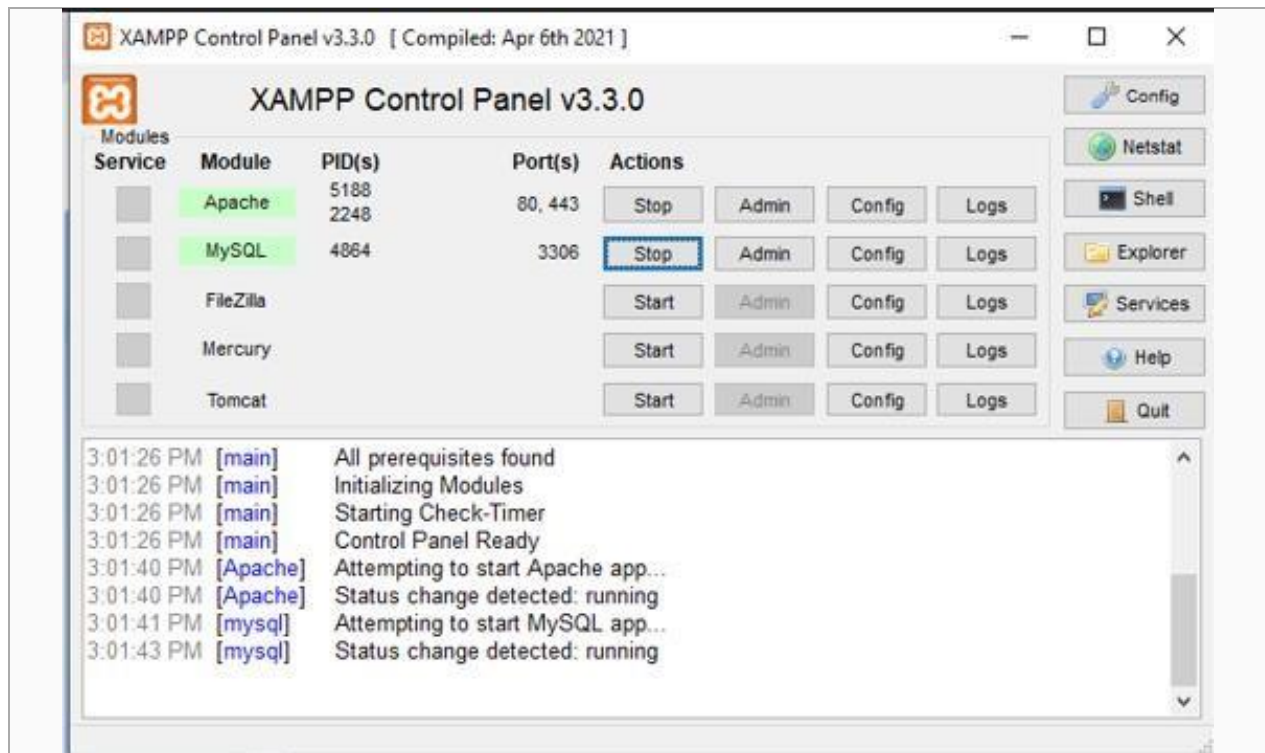


Fig 2: Apache and MySQL Running — XAMPP Control Panel showing both Apache and MySQL with a green “Running” status.

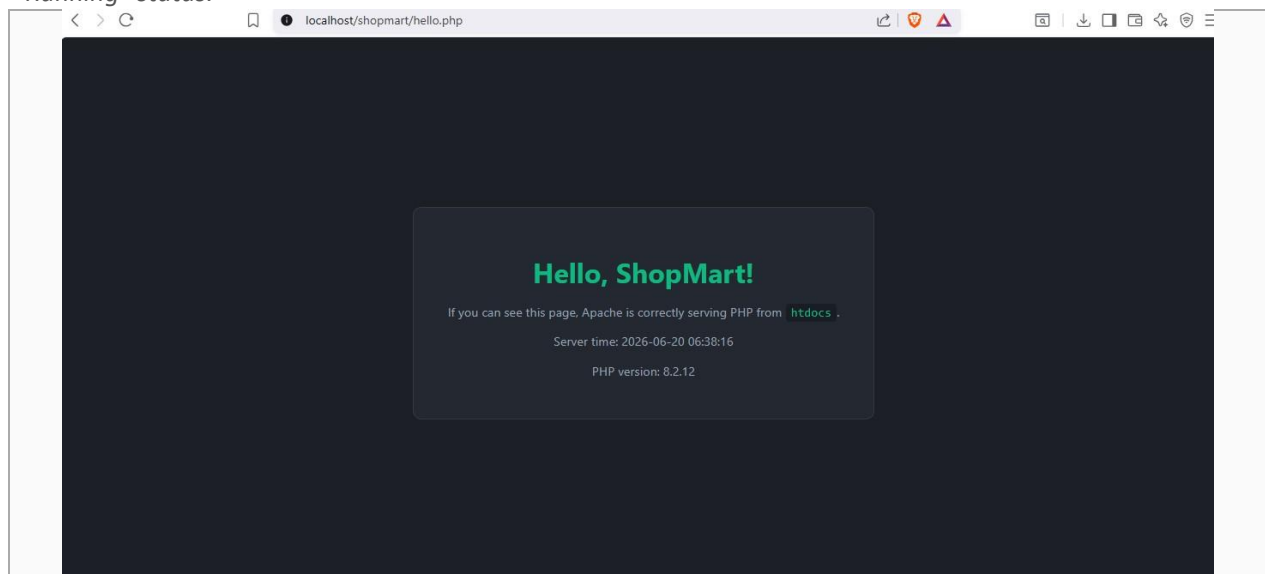


Fig 3: Hello World PHP Test — Browser showing *hello.php* with the live server time and PHP version, confirming PHP is executing correctly.

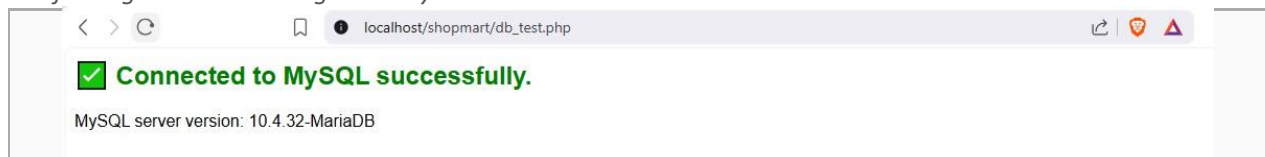


Fig 4: Database Connection Test — *db_test.php* showing a successful MySQL connection message.

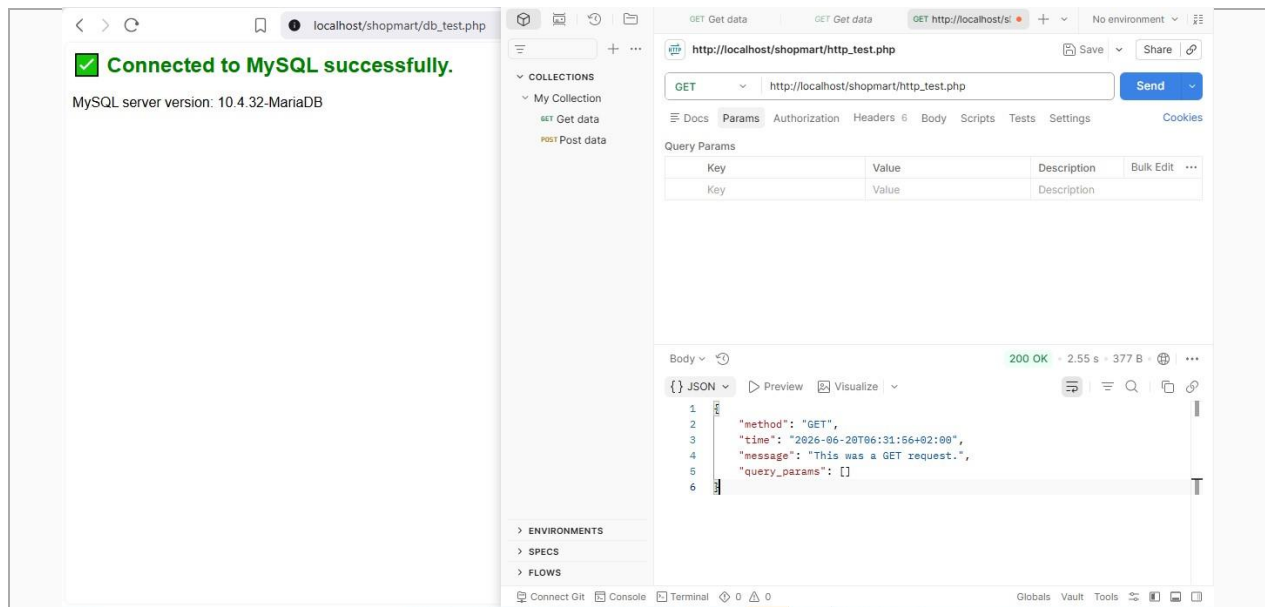


Fig 5: HTTP Test in Postman — A Postman GET request to `http_test.php` returning the expected JSON response.

Student Reflection (Maximum 100 Words)

Honestly this week was just about making sure the basics actually worked before touching anything else. Once `hello.php` showed the right server time I knew Apache and PHP were talking to each other properly, and `db_test.php` confirmed MySQL was reachable too. Testing `http_test.php` in Postman helped the most though — actually seeing the method, headers, and body PHP receives made the whole request/response thing finally click instead of just being theory. This matters for ShopMart because literally every feature I build later, forms, logins, all of it, depends on this same connection working.

Tools Used

- XAMPP — local Apache web server, MySQL database, and PHP runtime
- Postman — sending GET/POST requests and inspecting responses
- Web Browser — verifying localhost and viewing PHP output

Week 2: Wireframes and GUI Design

User Interface Planning and System Design

Title

User Interface Planning and System Design

What Was Built

Before any backend code was written, the ShopMart interface was planned. Low-fidelity wireframes were produced for the four core screens — Login, Dashboard, Homepage, and Product page —

focusing purely on layout and structure rather than colors or final styling, using the standard greyscale box-and-label wireframing convention. A higher-fidelity GUI concept was then produced to define the actual look and feel: a dark charcoal background with emerald green as the primary accent color, used consistently for buttons, links, and highlights.

Required Evidence

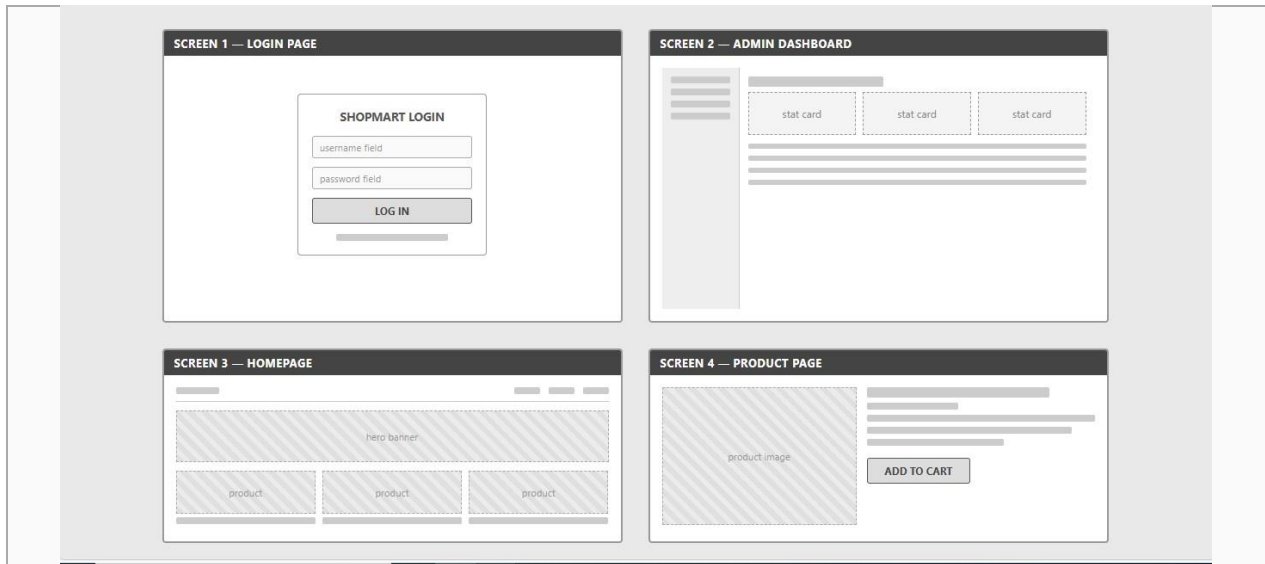


Fig 1: Wireframes — Login, Dashboard, Homepage, Product Page — *Low-fidelity wireframe layouts showing the four core ShopMart screens in greyscale box-and-label form.*

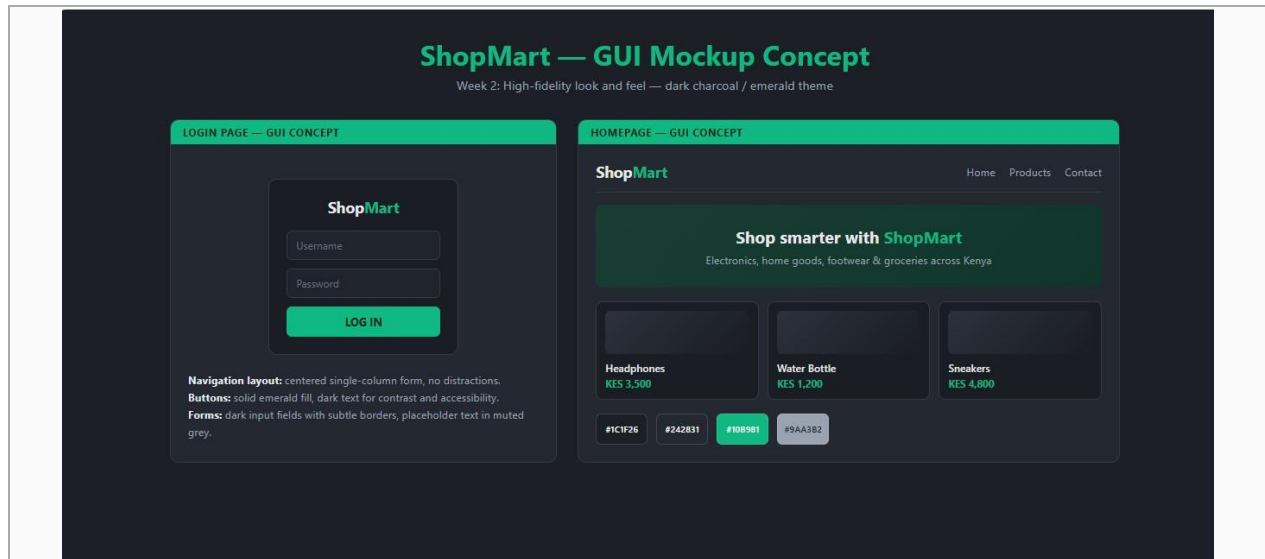


Fig 2: GUI Concept — Color Palette and Login Mockup — Higher-fidelity mockup showing the dark charcoal / emerald green color scheme applied to the Login screen.

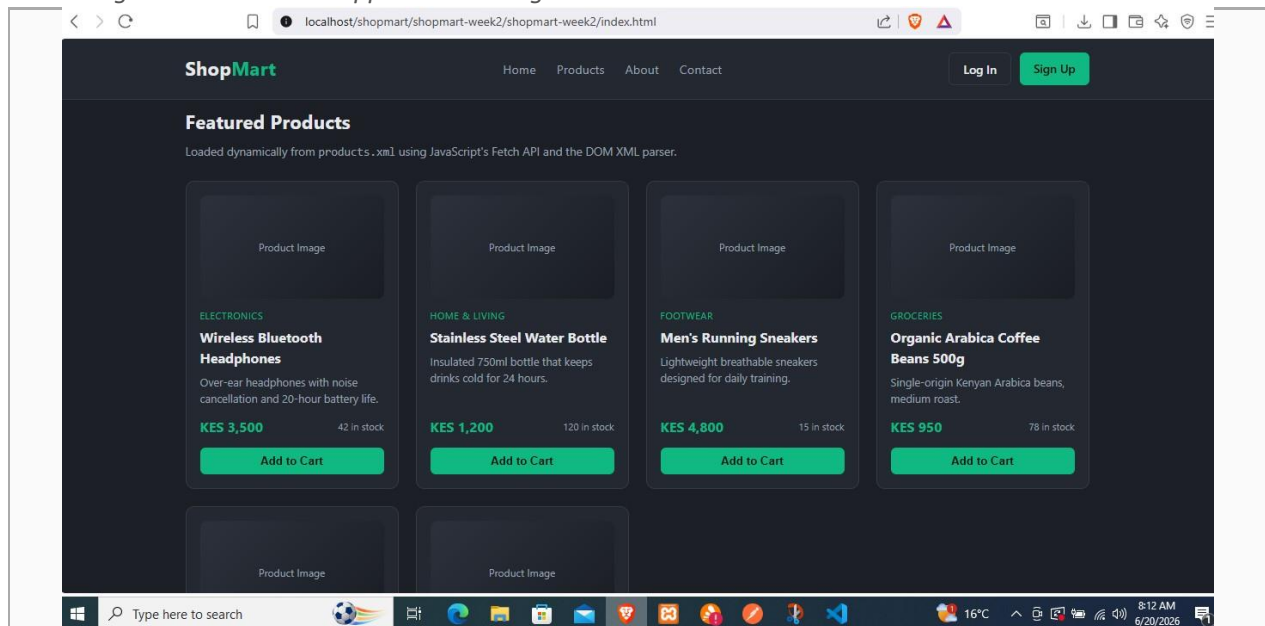


Fig 3: GUI Concept — Homepage Mockup — Higher-fidelity mockup showing the homepage hero section and product card layout in the final color scheme.

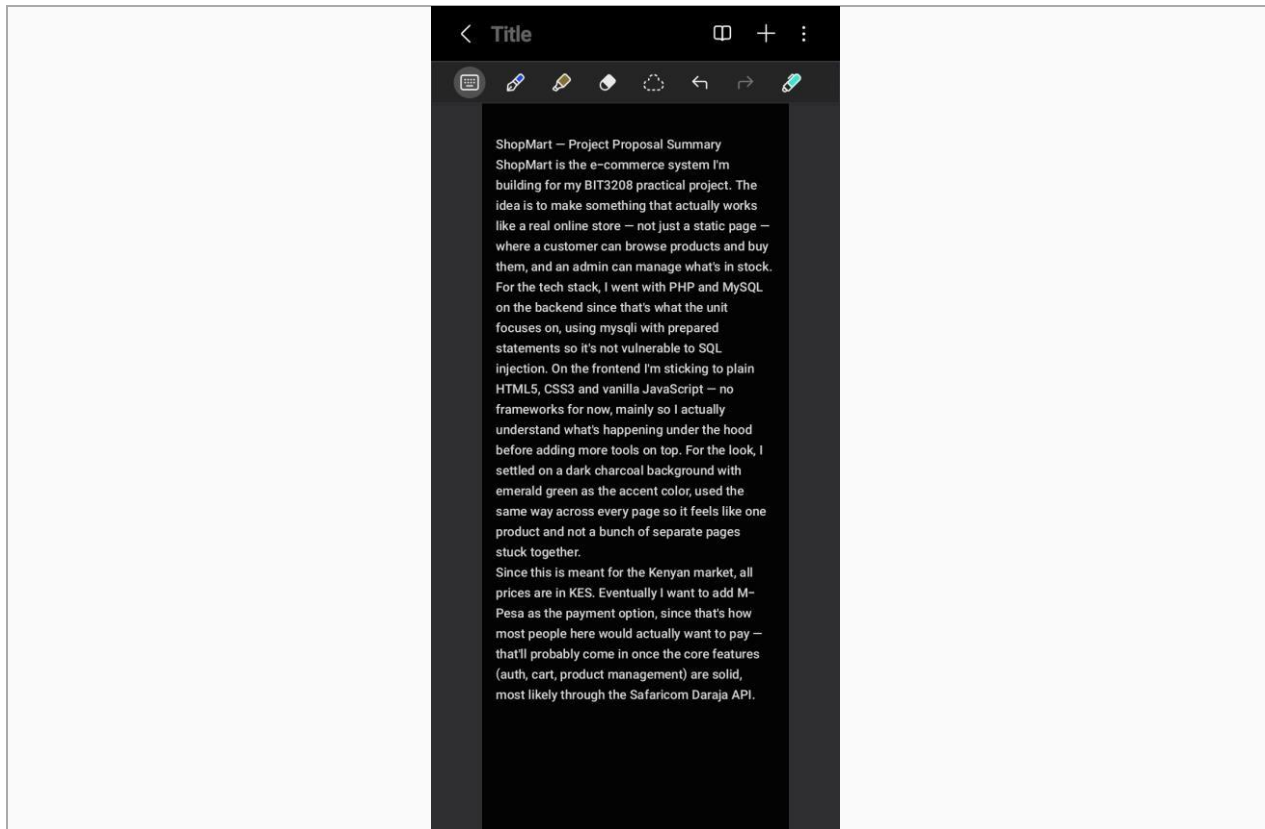


Fig 4: Project Proposal Summary — *Written summary outlining ShopMart's purpose, chosen technologies (PHP, MySQL, HTML5/CSS3/JS), and planned M-Pesa integration.*



Fig 5: Folder Structure Planning — Planned folder structure showing one directory per week (Week1–Week5) plus a Documentation folder, per the version-control guidance for this unit.

Student Reflection (Maximum 100 Words)

Planning the UI before writing any backend code actually made the later weeks way easier to deal with. Sketching the Dashboard wireframe first helped me figure out what the admin side actually needed, and picking the emerald-on-charcoal colors early meant I didn't have to redesign anything later. The biggest thing I had to remind myself was to keep the wireframes plain — I kept wanting to add color too early before the structure was even confirmed. This matters for ShopMart because having a clear plan saves me from redoing work once the real PHP and database stuff comes in.

Tools Used

- HTML/CSS (in place of Figma) — used to produce both the wireframes and the GUI mockup
- Browser — viewing and screenshotting both deliverables

Week 3: JavaScript and PHP Basics

Frontend Interaction and Backend Foundations

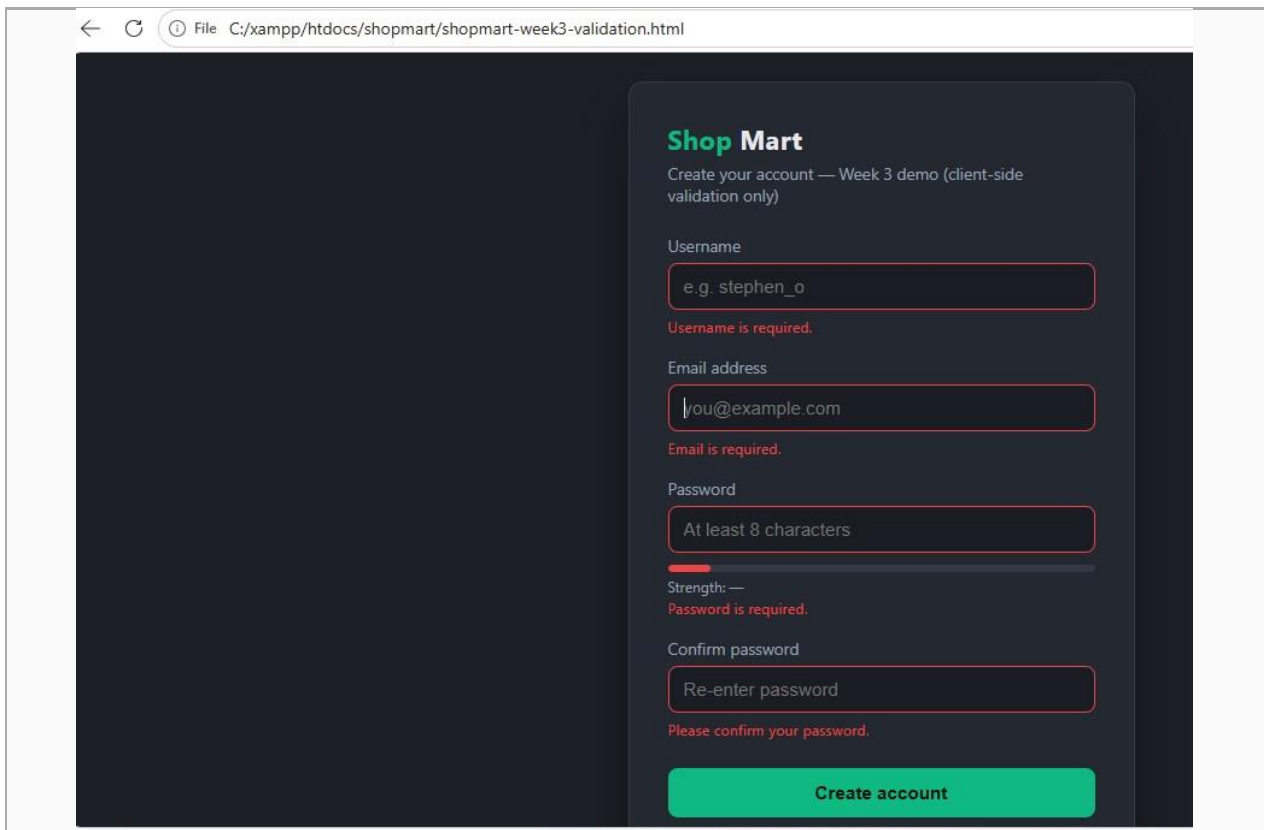
Title

Client-Side Form Validation for ShopMart Registration Using JavaScript

What Was Built

A standalone registration form for ShopMart (username, email, password, confirm password) was built using HTML5, CSS3, and vanilla JavaScript. The form runs entirely in the browser at this stage — it does not yet connect to PHP or MySQL, since backend integration was scheduled for the following weeks. The focus this week was DOM manipulation: reading user input, validating it on every keystroke, and updating the interface live without reloading the page.

Required Evidence



The screenshot shows a web browser window with the address bar displaying "File C:/xampp/htdocs/shopmart/shopmart-week3-validation.html". The main content area features a dark-themed registration form titled "Shop Mart" with the subtitle "Create your account — Week 3 demo (client-side validation only)". The form includes four input fields: "Username" (empty, with a red error message "Username is required." below it), "Email address" (containing "you@example.com", with a red error message "Email is required." below it), "Password" (containing "At least 8 characters", with a red error message "Password is required." below it), and "Confirm password" (containing "Re-enter password", with a red error message "Please confirm your password." below it). A green "Create account" button is positioned at the bottom of the form.

Fig 1: JavaScript Form Validation — Empty username field showing the live red error message generated by `validateUsername()`.

← ↻ ⓘ File C:/xampp/htdocs/shopmart/shopmart-week3-validation.html

Shop Mart

Create your account — Week 3 demo (client-side validation only)

Username

Username is required.

Email address

Email is required.

Password

Strength: Good

Confirm password

Please confirm your password.

Create account

Fig 2: Password Strength Checker — Password field showing the strength meter moving from “Weak” (red) to “Strong” (green) as a longer password with symbols is typed.

Shop Mart
Create your account — Week 3 demo (client-side validation only)

Username

Username is required.

Email address

Email is required.

Password

Strength: Good

Confirm password

Passwords do not match.

Create account

Fig 3: Confirm-Password Mismatch — *Confirm Password field showing the “Passwords do not match” error when the two password fields differ.*

Fig 4: All Fields Valid — Completed form with all fields outlined in green and the success banner confirming the data is ready for backend submission.

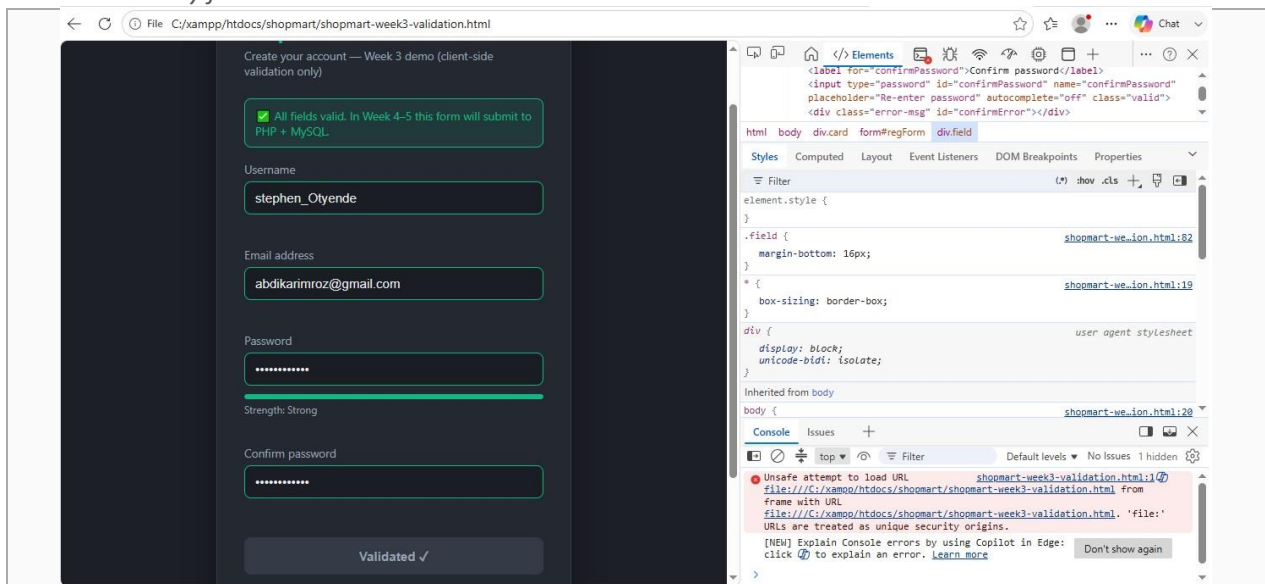


Fig 5: Browser DevTools — DOM Inspection — Chrome DevTools Elements panel showing the valid/invalid classes being added to input elements live, confirming DOM manipulation is working as coded.

Student Reflection (Maximum 100 Words)

This week showed me how much JavaScript can do before any data even reaches a server. The registration form checks the username, email format, password strength, and confirm-password match instantly using DOM manipulation, no reload needed. The password-strength meter gave me the most trouble since it has to score multiple character types at once instead of just one simple check. This matters for ShopMart because catching errors on the frontend means fewer pointless requests hitting the PHP backend once the form actually connects to MySQL in the next weeks.

Tools Used

- VS Code — writing and editing HTML/CSS/JavaScript
- Google Chrome + DevTools — testing validation logic and inspecting the DOM
- Browser (offline, no server required) — running the standalone HTML file

Week 4: User Authentication

Basic Login System Connected to MySQL

Title

Basic PHP Login System with MySQL Database Connection

What Was Built

A basic registration and login system was built for ShopMart using PHP and MySQL, connected with the mysqli extension. The implementation was kept deliberately simple at this stage: passwords are hashed with bcrypt using `password_hash()`, and prepared statements are used throughout to prevent SQL injection. On successful login, PHP starts a native session and stores the user's id and username, which a protected dashboard page checks before allowing access. Login attempts are verified using `password_verify()` against the stored hash, never by comparing plain text.

Required Evidence



Fig 1: Registration Form & Success Message — *register.php* showing a filled-in registration form and the success message after submission.

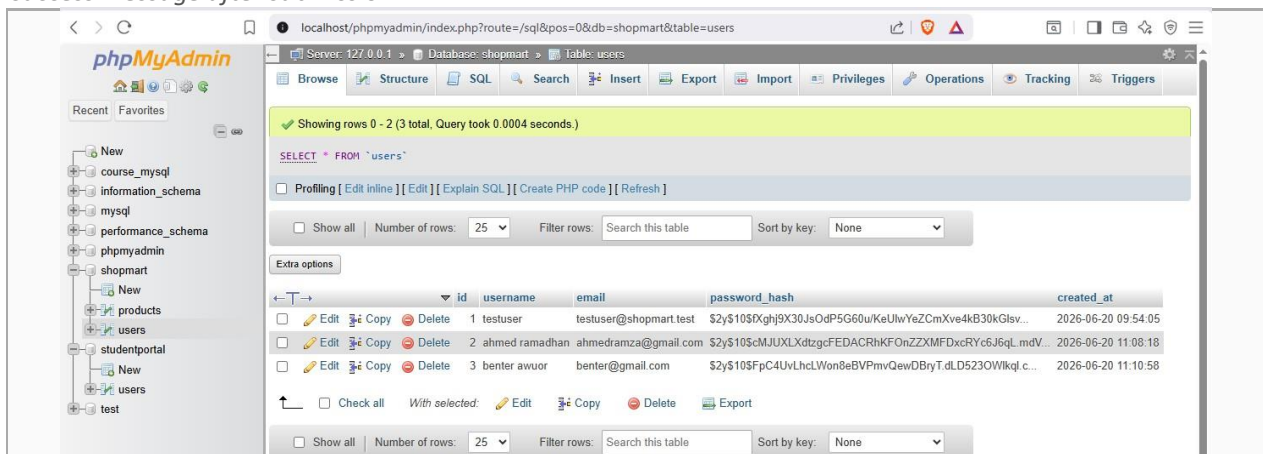


Fig 2: Hashed Password in Database — *phpMyAdmin* view of the *users* table showing the *password_hash* column stored as a bcrypt hash (starting with `$2y$10$`), never as plain text.

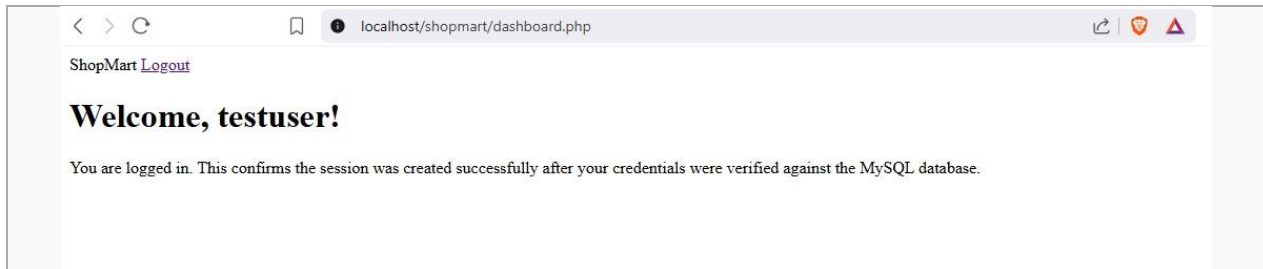


Fig 3: Successful Login & Dashboard — *dashboard.php* after logging in, showing a welcome message pulled from `$_SESSION['username']`.



Fig 4: Invalid Login Attempt — *login.php* showing the “Invalid username or password” message when incorrect credentials are submitted.

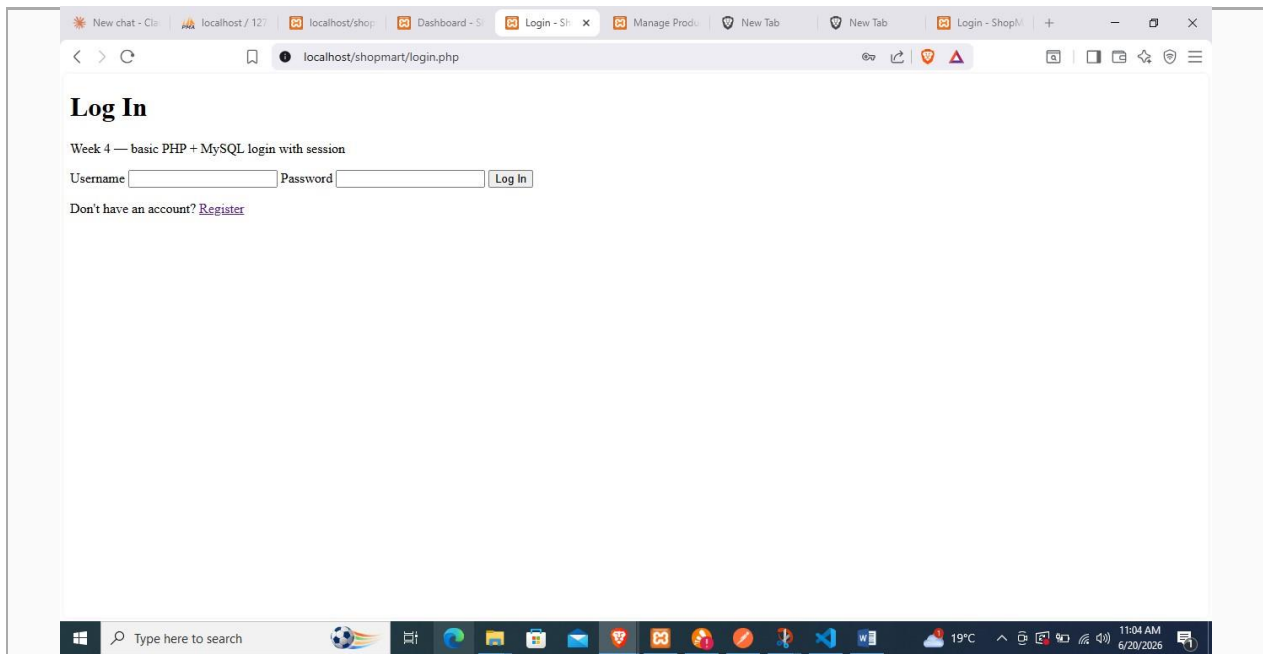


Fig 5: Logout Clearing the Session — Browser redirecting back to *login.php* after clicking Logout, confirming the session was destroyed.

Student Reflection (Maximum 100 Words)

This was the first week the frontend actually talked to a real backend. Using mysqli with prepared statements made it obvious why you should never just concatenate raw SQL — the placeholders keep whatever the user types separate from the actual query. The part that gave me the most

trouble was getting `password_hash()` and `password_verify()` working together properly, since the hash only gets created once at registration and should never be regenerated at login. This matters for ShopMart because it proves the whole login system actually works end to end before I add any more security on top.

Tools Used

- XAMPP — local Apache web server and MySQL database
- phpMyAdmin — inspecting the users table and verifying stored data
- VS Code — writing the PHP scripts

Week 5: Database Components

Full CRUD Operations for Product Management

Title

Product Management Module with Full CRUD Operations (Create, Read, Update, Delete)

What Was Built

A complete product management module was built for ShopMart, allowing a logged-in user to Create, Read, Update, and Delete product records stored in MySQL. The module reuses the session check first built in Week 4, so only an authenticated user can reach these pages. All four operations use mysqli prepared statements with bound parameters, which keeps user input separated from the SQL query text and protects against SQL injection. The product list refreshes automatically after every Create, Update, or Delete action and displays a confirmation banner so the result of each action is clear.

Required Evidence



Fig 1: Product List (Read) — *index.php* showing the products table loaded from MySQL, with Edit and Delete links for each row.



Fig 2: Adding a Product (Create) — *create.php* form filled in with a new product's details, and the resulting "Product created successfully" message.

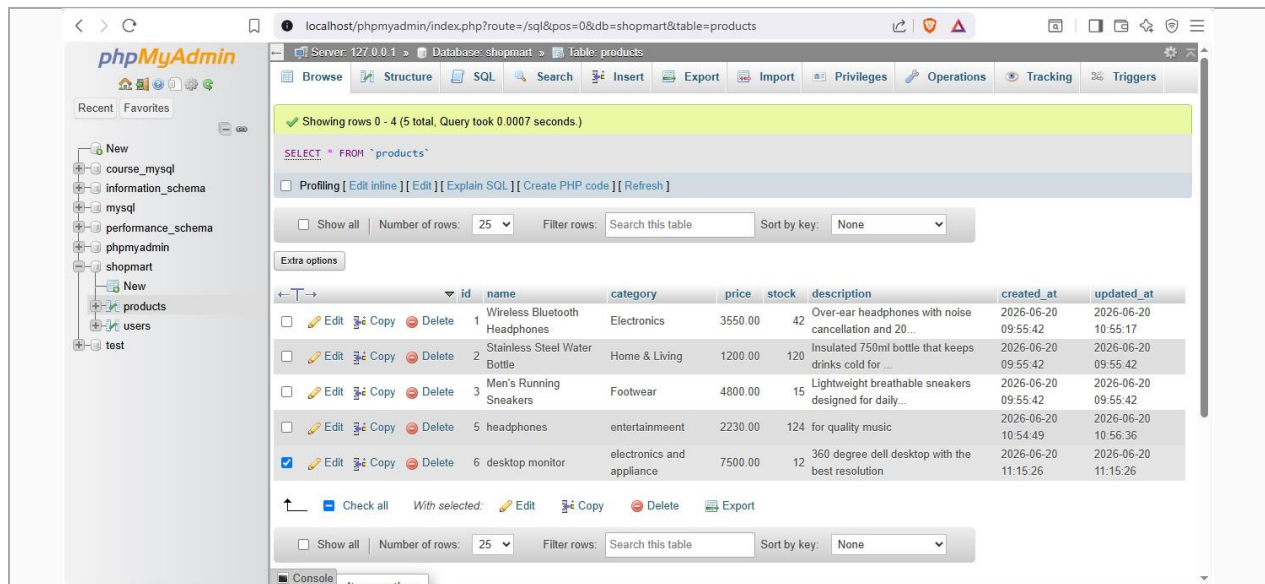


Fig 3: New Product Confirmed in phpMyAdmin — phpMyAdmin view of the products table showing the newly inserted row matching the form data.



Fig 4: Editing a Product (Update) — edit.php pre-filled with an existing product's data, with the price or stock field changed, and the “Product updated successfully” confirmation.

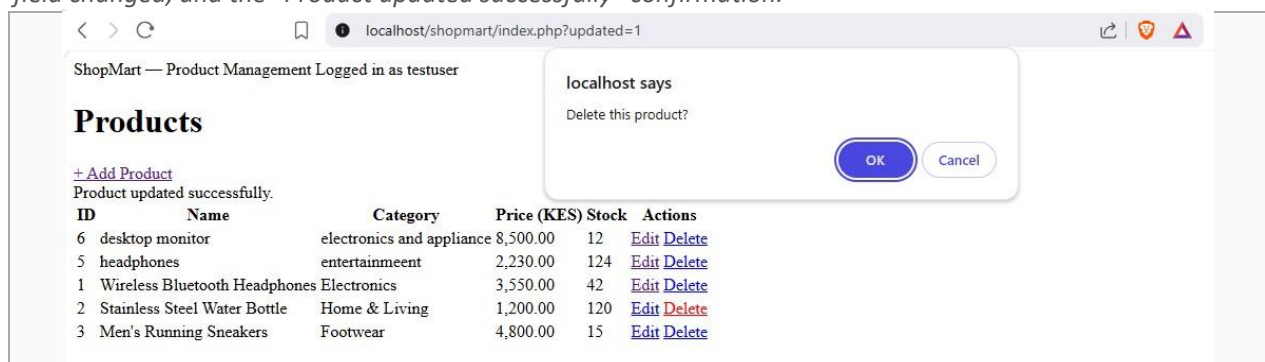


Fig 5: Deleting a Product (Delete) — The confirm() dialog before deletion, followed by the updated product list showing the row removed and the “Product deleted” message.

Student Reflection (Maximum 100 Words)

This week pulled everything from the earlier weeks together into something an admin could actually use — a working product catalog connected to MySQL. The UPDATE query gave me the most trouble

since the bound parameter types have to match exactly or mysqli just fails silently with no real error message.

Reusing the Week 4 session check instead of rewriting it also showed me why splitting things into small included files actually saves time later. This matters for ShopMart because product management is basically the core of the admin side — everything else depends on this data being right.

Tools Used

- XAMPP — local Apache web server and MySQL database
- phpMyAdmin — verifying inserted, updated, and deleted product records
- VS Code — writing the PHP CRUD scripts